

Custom Marker Detection & Extraction

Approach — Alemeno Frontend Internship Assignment

1. Summary

The app is a React Native (Android) project that opens the back camera, runs every frame through an OpenCV pipeline on a worklet thread, and collects 20 orientation-corrected, tightly-cropped 300×300 marker images from 20 different frames. The detector targets the published geometry of **Marker 1** — a 140×140 black-framed square with a single 20×20 anchor square in a corner of the inner white area — and is validated against all 7 provided test images plus a synthetic stress test (rotations, perspective skews, low light).

Validation result on the official test set: **7 of 7 pass** (3 correct markers detected, 4 incorrect markers rejected).

2. Tech stack — and why each piece

Layer	Choice	Why
Framework	React Native 0.76 (CLI)	Required by spec; CLI lets us link OpenCV native code
Camera	react-native-vision-camera v4	Industry standard, supports frame processors via worklets
Image processing	react-native-fast-opencv v0.4	OpenCV bound directly into vision-camera worklets — no thread hop per frame
Frame downscaling / cropping	vision-camera-resize-plugin	GPU-accelerated crop + resize before CV pipeline
Worklet bridge	react-native-worklets-core	Required by vision-camera for JS↔worklet communication
State / nav	Local React state	Two-screen flow doesn't justify a router

The single most important choice was **react-native-fast-opencv**. Conventional React Native CV libraries either (a) round-trip every frame through the JS bridge, costing 30–80 ms per frame, or (b) make you write your own native module from scratch. **fast-opencv** binds OpenCV directly into vision-camera's worklet runtime, so each frame stays on the camera thread from capture through to the final extracted patch. On a mid-range Android phone this is the difference between ~5 fps and ~25 fps — comfortably inside the 3000 ms scan-to-result budget the assignment lays out.

3. Detection pipeline

Each camera frame goes through this pipeline:

- ① **Centre-cropped square downscale.** The phone streams 4K (3840×2160). We crop a centred square of side $\min(\text{width}, \text{height}) \approx 2160$ px (within the assignment's 2000–3000 px requirement) and downscale to 720×720 in a single GPU pass via **vision-camera-resize-plugin**. Detection runs on the 720×720 view — accuracy is unaffected for markers ≥ 60 px in that view, and OpenCV's **findContours** is roughly $O(\text{area})$, so analysing 0.5 MP instead of 4.7 MP gives a ~10× speed-up.
- ② **Grayscale + blur.** **cvtColor** → **GaussianBlur** (5×5). Reduces sensor noise without erasing sharp marker edges.
- ③ **Adaptive threshold.** **adaptiveThreshold** with a 21×21 Gaussian window, offset 10, **THRESH_BINARY_INV**. Far more robust than Otsu under uneven mobile lighting (e.g. one side of the marker is shadowed).

- ④ **Find contours.** findContours with RETR_EXTERNAL, CHAIN_APPROX_SIMPLE.
- ⑤ **Filter to square candidates.** For each contour passing the area threshold ($\geq 60 \times 60$ px²): (1) **Convex hull first.** When a marker is captured from a laptop screen, sub-pixel rendering and moiré make the raw contour edges jagged, so a single approxPolyDP call typically lands on tiny zigzag corners instead of the four real ones. Running convexHull first strips those concave noise points. (2) **Multi-epsilon approxPolyDP.** Try epsilons {0.02, 0.03, 0.04, 0.05, 0.06, 0.08} $\times$ perimeter, accept the first that yields exactly 4 vertices. (3) **Squareness check.** Order the 4 corners (TL, TR, BR, BL via the standard sum/diff trick), require the shortest side ≥ 60 px and the longest/shortest ratio ≤ 2.5 .
- ⑥ **Perspective warp.** getPerspectiveTransform + warpPerspective to a fixed 300 $\times$ 300 patch.
- ⑦ **Validate the patch** (see §4) — the most important step.
- ⑧ **Orientation correction.** cv::rotate by 0/90/180/270° based on which corner the anchor was found in (see §5). 90° increments use no pixel resampling, so the corrected patch keeps full 300 $\times$ 300 sharpness.
- ⑨ **Encode.** Final 300 $\times$ 300 BGR Mat $\rightarrow$ base64 PNG $\rightarrow$ handed back to the JS thread via a Worklets.runOnJS dispatcher.

4. The patch validator

After perspective-warping a candidate to a normalized 300 $\times$ 300 patch, the validator decides whether it's actually a Marker 1. Getting this right is what separates the 4 "incorrect" test samples (which all share the exact same outer frame) from the 3 "correct" ones.

Checks, in order:

- **Border ring is ink.** Sample a 5%-wide ring along the patch edges. Average ink ratio must be ≥ 0.55 . Confirms the outer black frame exists. The threshold is loose because approxPolyDP corners often land slightly inside the actual frame edge, so the ring picks up a few interior white pixels — anything well above 0.5 still implies a dominant outer black frame, while values around 0.3 (which a non-marker quadrilateral typically produces) get rejected. Dashed-frame samples (Marker 2) also fail this check because the gaps drop the ratio to ~ 0.4 .
- **Centre is not solid black.** The centre 50% region's ink ratio must be ≤ 0.85 . Allows decorative content (the spec explicitly lets information be encoded inside) but rejects a fully filled solid square.
- **Find filled square contours anywhere in the patch.** RETR_LIST find-contours on the binary patch. For each contour: bounding box, aspect ratio, and density (filled-mask area $\div$ bbox area).
- **Bucket each square contour.** A contour is treated as an *anchor* candidate iff density ≥ 0.85 AND aspect ≤ 1.25 (filled, square-shaped) AND size 9–21% of N AND in a corner zone AND clear of the outer frame. Wrong size, wrong position, or touching the frame $\rightarrow$ mark as a *disqualifying* blob.
- **Final decision.** Pass iff exactly 1 anchor candidate AND 0 disqualifying blobs. This single rule rejects all four incorrect samples — anchor too big, anchor centred, anchor touching the frame, or any combination thereof.

Why filled-vs-outline density matters. Two of the three correct test images contain large illustrations (a pig, a dog, a monkey) inside the marker. Their outline contours have density 0.3–0.7 (hollow line art, not filled blocks). By only treating high-density (≥ 0.85) square contours as anchor candidates or disqualifiers, the validator ignores decorative content while still catching wrong-shaped anchors.

5. Orientation correction

Once the validator finds the anchor square, its corner location reveals which 90° rotation is needed to bring it to the canonical top-left. `cv::ROTATE_90_CLOCKWISE` maps old position → new as TL→TR, TR→BR, BR→BL, BL→TL — so to bring an anchor at corner X to TL, we need the rotation that moves X→TL:

Anchor location	Rotation applied	OpenCV flag
Top-left (canonical)	0°	—
Top-right	90° counter-clockwise	ROTATE_90_COUNTERCLOCKWISE
Bottom-right	180°	ROTATE_180
Bottom-left	90° clockwise	ROTATE_90_CLOCKWISE

OpenCV's `cv::rotate` handles 90° increments without any pixel resampling, so the corrected patch keeps its full 300×300 sharpness. The perspective warp in step ⑥ has already removed any non-90° rotation; this step only handles the remaining 4-way symmetry. (The mapping was originally inverted for TR/BL — anchors in those corners ended up in the bottom-right of the output instead of top-left. Fixed and verified.)

6. Validation against the official test set

A Python prototype (`tests/detect_prototype.py`) mirrors the JS algorithm using `cv2`. Running it against all 7 provided test images:

Test image	Expected	Result	Notes
Marker1-TestImage1-Correct.jpg	ACCEPT	✓ ACCEPT	Pig illustration, upright, anchor TL
Marker1-TestImage2-Correct.jpg	ACCEPT	✓ ACCEPT	Dog, marker rotated 45° (diamond)
Marker1-TestImage3-Correct.jpg	ACCEPT	✓ ACCEPT	Monkey, anchor in BR corner
Marker1-TestImage4-Incorrect.jpg	REJECT	✓ REJECT	Anchor touches inner frame
Marker1-TestImage5-Incorrect.jpg	REJECT	✓ REJECT	Anchor in dead centre
Marker1-TestImage6-Incorrect.jpg	REJECT	✓ REJECT	Anchor 2× too large + touching
Marker1-TestImage7-Incorrect.jpg	REJECT	✓ REJECT	Anchor 2× too large + centred

Score: 7 / 7. All three correct markers detected with the right orientation correction; all four incorrect markers rejected.

7. Synthetic stress test

Synthetic perturbations of each correct test image: rotations every 15° from 0° to 345°, perspective skews of 5/10/15/20%, and brightness reductions to 70/50/40% of original. 31 perturbations × 3 base images = 93 cases.

Score: 81 / 93 passed (87.1%). The 12 failures are all *graceful* — the detector outputs "no marker" rather than producing a false positive — and cluster around extreme conditions: rotations of 135°/315° clip the marker off the synthetic 500×500 canvas, skews of 15–20% are beyond what a user generates when holding the phone roughly level, and brightness ≤ 50% is well past reasonable indoor lighting. For the live use case the detector only needs to succeed on 20 frames out of the hundreds the user scans past.

8. Speed

The assignment asks for under 3000 ms scan-to-result. Three design choices drive the speed:

- **Worklet-resident OpenCV.** Frames never cross the JS bridge during processing. react-native-fast-opencv runs OpenCV directly inside the vision-camera worklet, so the only data that crosses threads is the final 300×300 PNG (~30 KB).
- **Aggressive downscaling.** The camera streams 4K (3840×2160) but detection runs on a 720×720 centre-cropped view. findContours is roughly $O(\text{area})$, giving a ~10× speed-up with no measurable accuracy loss for markers ≥ 60 px.
- **Tight throttling.** A unique marker is accepted into the result list once every 80 ms. With 20 captures $\times$ 80 ms $\approx$ 1.6 s plus ~500 ms camera warm-up, the total scan-to-result lands around 2 s — well under the 3000 ms target.

9. Known limitations

- Heavy perspective skew ($>15^\circ$) or very low light will sometimes miss valid markers — see the 12 stress-test failures.
- The first frame after launch can take ~500 ms longer than subsequent frames because OpenCV JIT-compiles inner loops.
- Tested on Android only. The RN code would work on iOS but I haven't built/signed for iOS.
- If two valid markers appear in the same frame, only the highest-scoring one is captured (by design — avoids the user accidentally getting two copies of the same physical marker counted as different captures).